PROPEL Technologies and Tools

PROPEL | Revised 5 November 2026

Architecture

Browser → Vite frontend → Django REST API → SQLite

Evidence adapters and assessments → reliability feedback → decayed competency state → confidence gates → exact catalog planner → updated browser report. Job discovery feeds an editable target profile into the same calculation loop.

Frontend

HTML5, CSS3 and JavaScript ES modules retain the existing Propel design. Vite builds the app. Three.js renders the assembled silver and navy rocket, blue bands and cool exhaust; its desktop position is shifted left. GSAP supports existing page transitions. New twin controls show target requirements, evidence entry, confidence components, estimated plans, assessments and exportable JSON reports. Jobs can be selected as target roles.

Backend and persistence

Python 3.10 or newer, Django 5.2, Django REST Framework 3.16, django-cors-headers, Requests and BeautifulSoup retain the supplied stack. SQLite stores accounts, listings, evidence, reliability adjustments, target profiles, snapshots, assessment attempts and activity completions. pypdf and python-docx add proper PDF and DOCX resume extraction. Exact tested dependency versions are pinned in backend/requirements.txt. Optional python-jobspy is separated into requirements-optional.txt.

Calculation modules

api/engine.py implements source weights, true age decay, confidence decomposition, boundary-aware skill matching and exact finite catalog optimization. api/service.py maintains persistent account feedback and snapshots. api/twin_views.py validates evidence, parses files, imports public coding signals and scores assessment answers. api/assessments.py contains the server-side knowledge-check bank. api/catalog.json stores competencies, role templates and learning resources. The revised IDF defines the equations and limits.

API groups

Public: GET /api/twin/catalog/, GET /api/twin/sample/, POST /api/twin/parse-target/, and GET /api/jobs/.

Authenticated: GET /api/twin/; POST /api/twin/target/, /evidence/, /resume/, /github/, /codeforces/, /assessment/start/, /assessment/<id>/submit/, and /activity/complete/. These suffixes are relative to /api/twin/.

POST /api/jobs/scrape/ requires sign in and has a one-minute per-account cooldown in the development cache. Existing authentication, contact, tracker and notification routes remain. Staff can enter reviewed evidence in Django admin.

Running and verification

Extract the ZIP. On macOS run start.command; on Windows run start-windows.bat. Python and Node.js/npm must already be installed. The launchers install dependencies, migrate a fresh database and serve the frontend at http://127.0.0.1:3000 and backend at http://127.0.0.1:8000. No accounts, private database or credentials are shipped.

For manual steps and test commands, see README.md. Thirty automated tests passed in the initial verification run, including comparison of the planner with exhaustive enumeration on small examples. Browser testing exercised saved evidence and contradiction feedback. The sample profile requires no account but still needs the backend.

Prototype boundaries

Source weights, decay rates, target defaults, confidence thresholds and activity gain estimates require calibration. Knowledge checks are unsupervised. Public GitHub and Codeforces handles do not prove account ownership. OCR, general natural-language parsing, platform ownership authentication, proctoring, background monitoring, deployment hardening and outcome validation are not included. Job portal availability can change; clearly labelled samples provide a demonstration fallback.